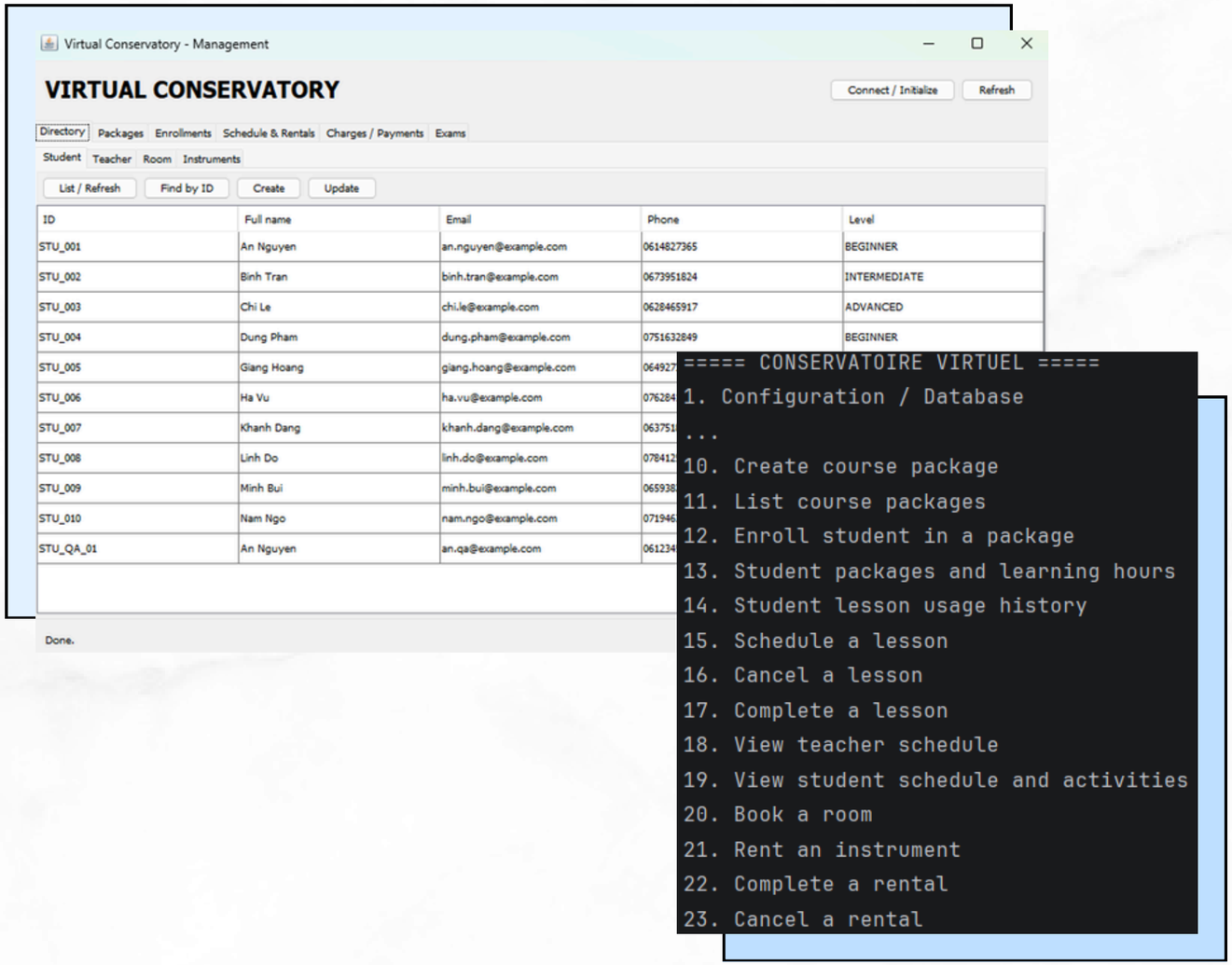
Conservatoire Virtuel

Music School Management System

Presented by:
Tien Vuong VU Quoc-Trung TUONG

Application



01

People

Student and Teacher

02

Learning

Packages and Lessons

03

School Resources

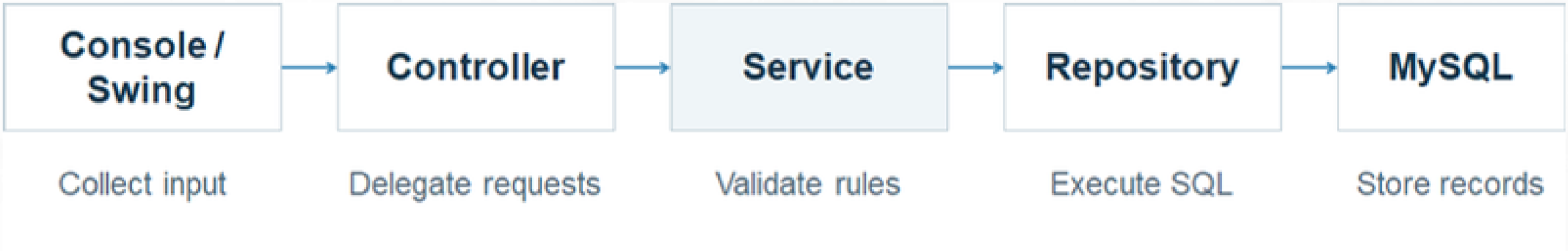
Rooms, Instruments and Rentals

04

Administration

Exams, Charges, Payments

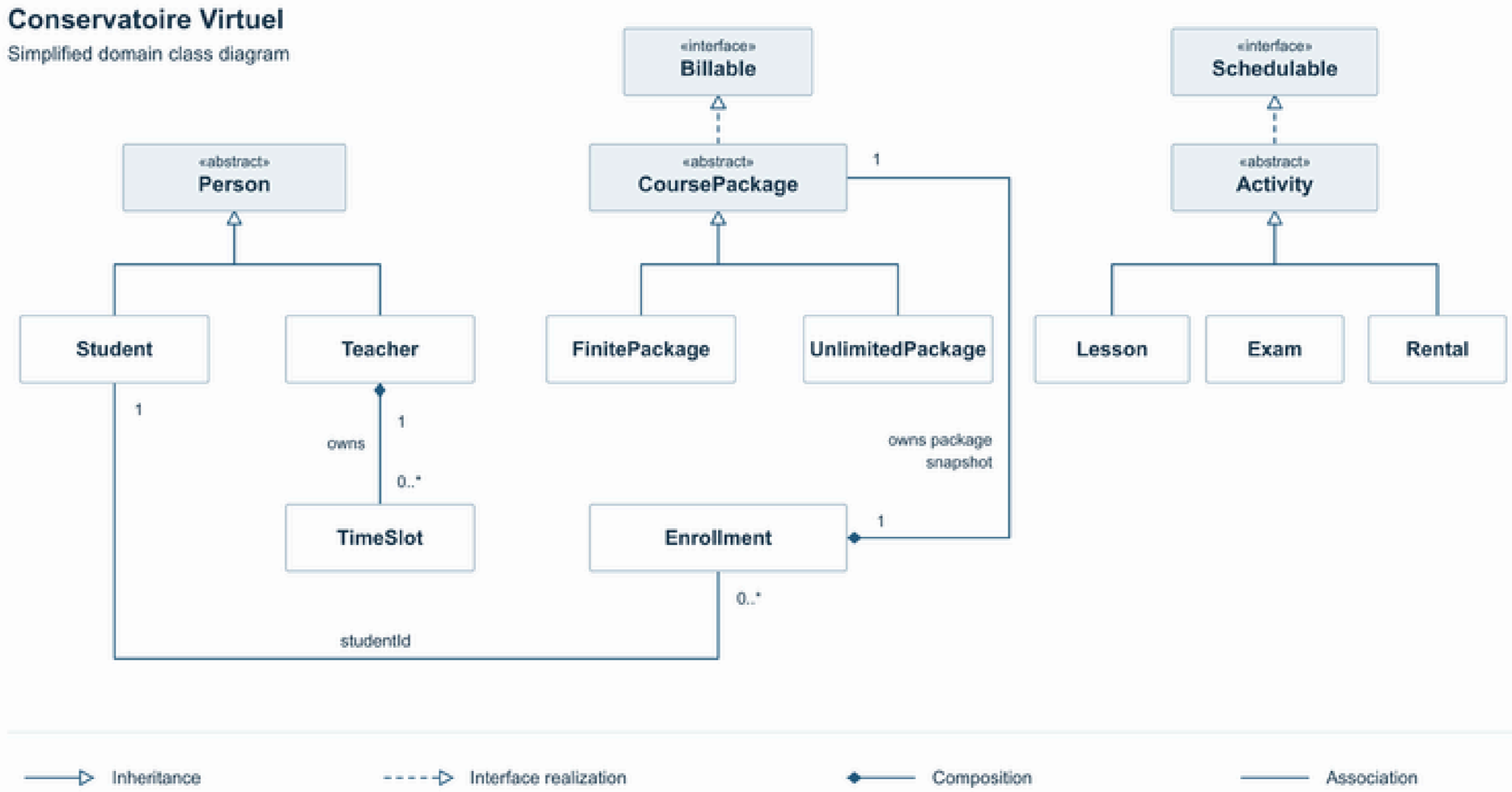
Backend Architecture



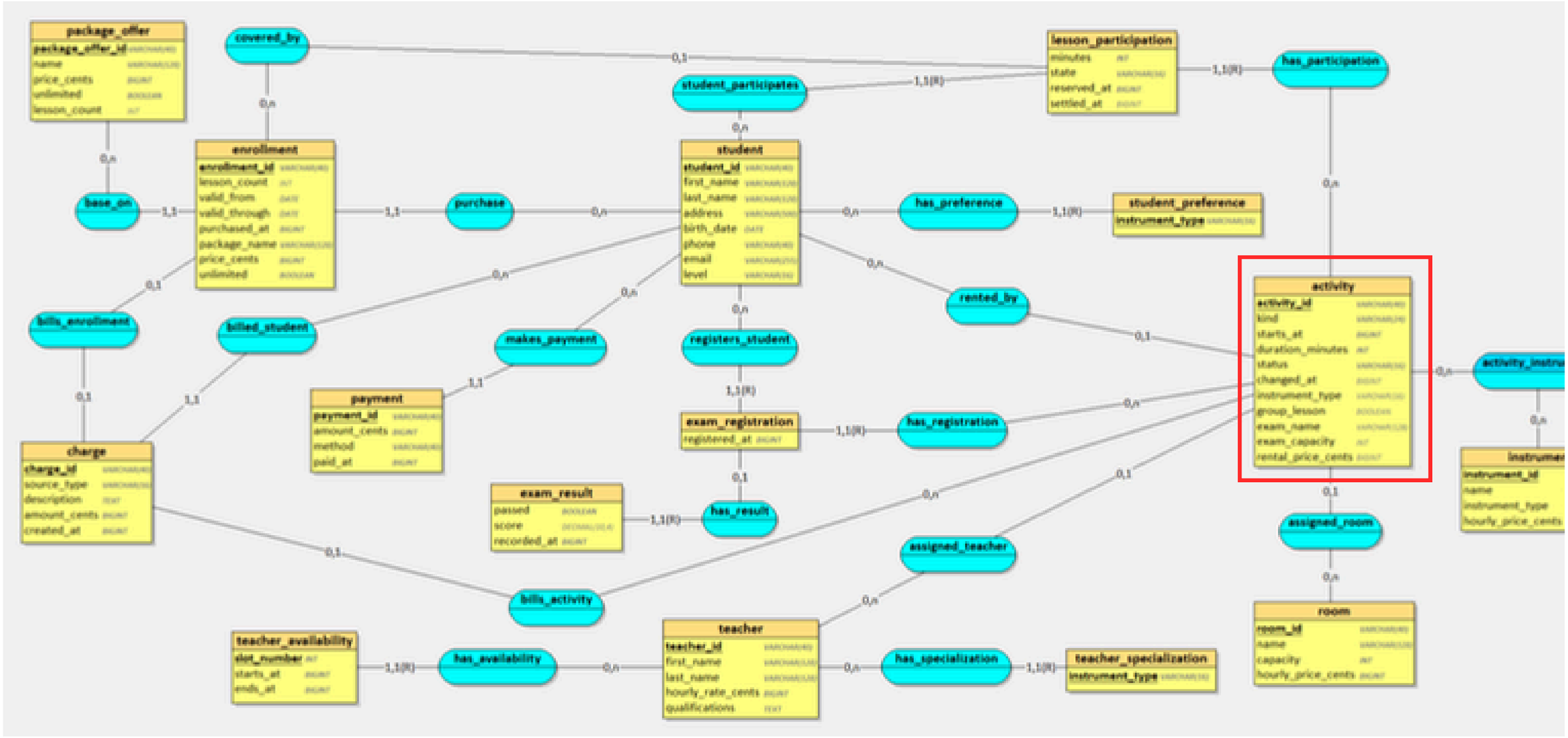
Domain models

Activity, Enrollment, Exam and CoursePackage contain shared state and behavior.

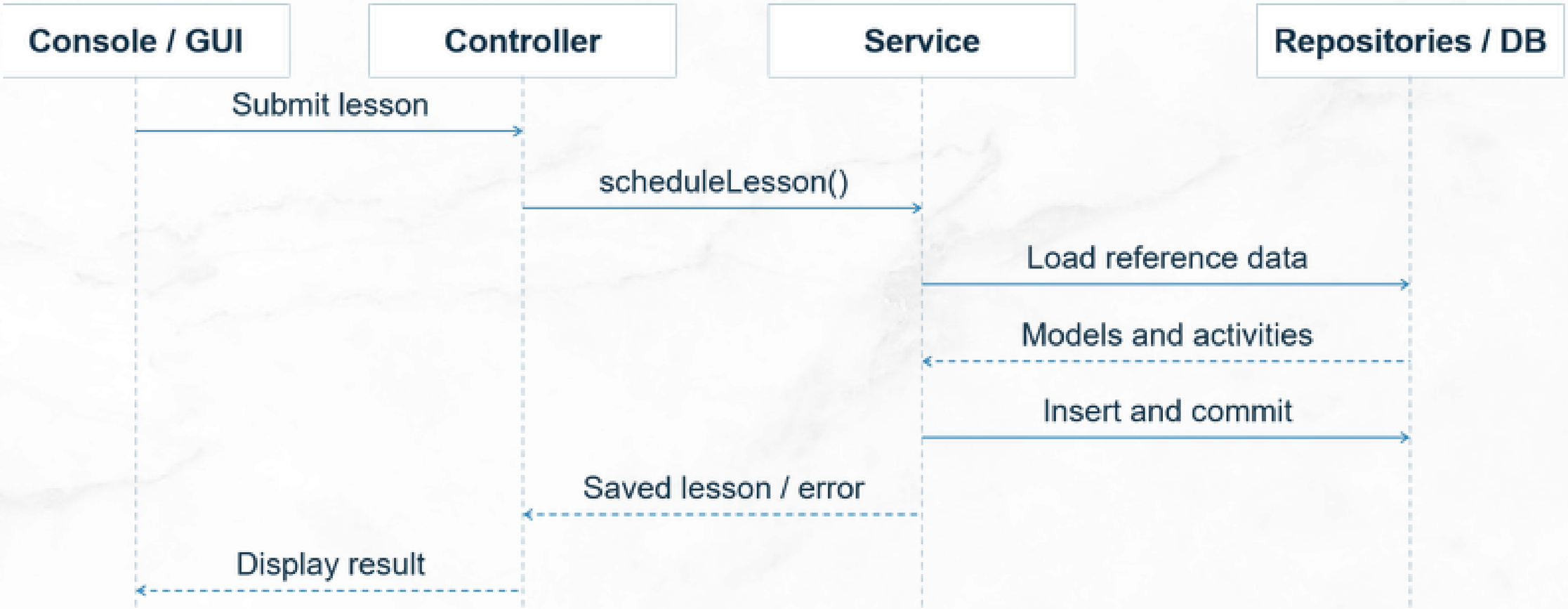
Class Diagram



Database ERD

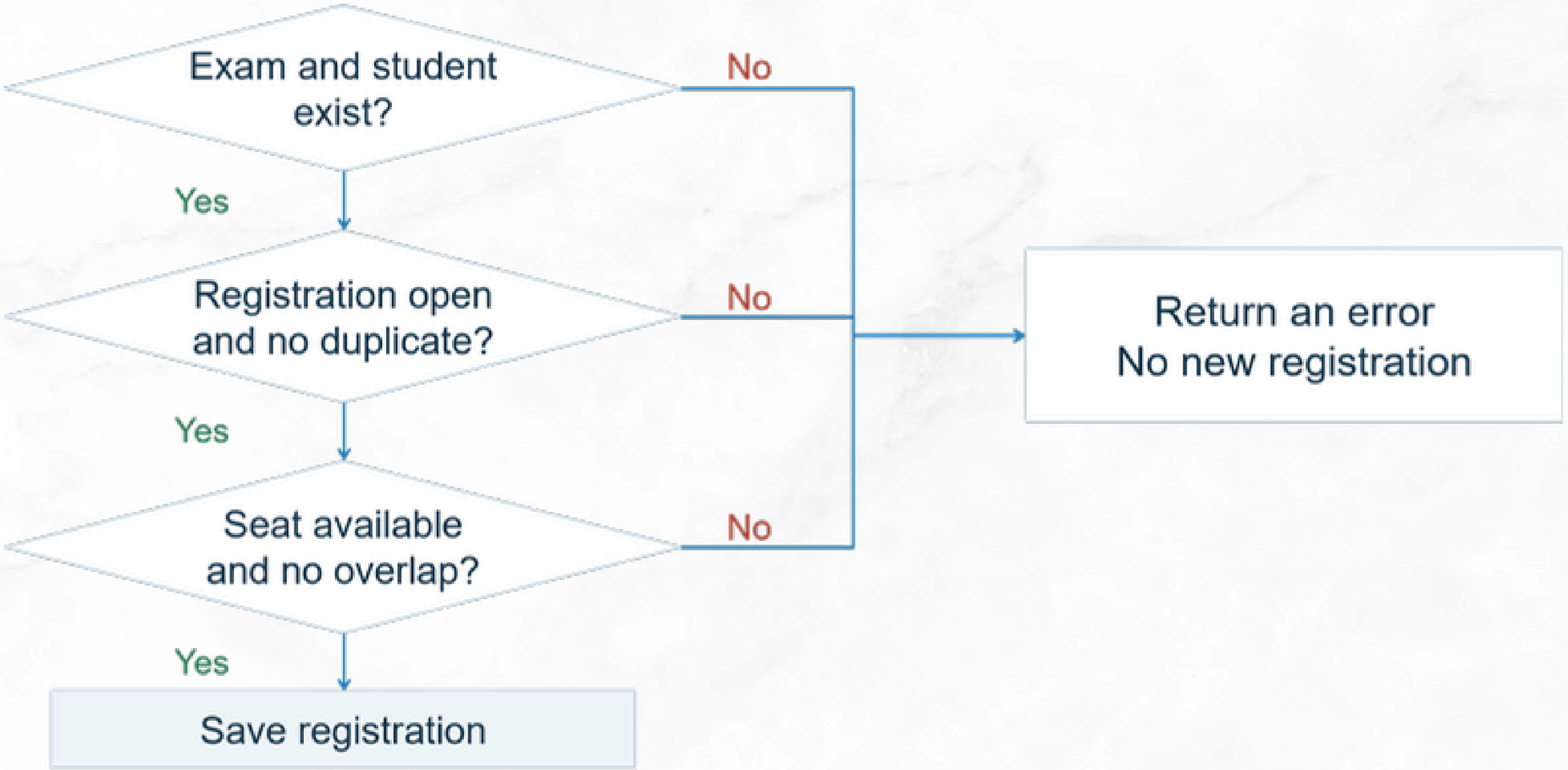


Squence Diagram (Scheduling)



Brief version of Squence Diagram (Scheduling)

Exam registration activity flow



Brief version of Activity Diagram (Exam Register)

Abstract classes



Interfaces

Billable

```
getPriceCents()
```

Schedulable

```
getStart()   getEnd()
```

List<CoursePackage>

```
for (CoursePackage p : packages) {
    System.out.println(p.isUnlimited());
}
```

FinitePackage: false

UnlimitedPackage: true

Inheritance

```java
public abstract class Person {  2 usages  2 inheritors
    private String id;  3 usages
    private String firstName;  3 usages
    private String lastName;  3 usages

public class Teacher extends Person{
    private final long hourlyRateCents;  3 usages
    private final String qualifications;  3 usages

public class Student extends Person {
    private final String address;
    private final LocalDate birthDate;
    private final String phone;
    private final String email;
    private final String level;
```

Composition

```
private final List<TimeSlot> availability = new ArrayList<>();  5 usages

public Teacher(String id, String firstName, String lastName, long hou
    super(id, firstName, lastName);
    this.hourlyRateCents = ModelChecks.price(hourlyRateCents);
    this.qualifications = ModelChecks.text(qualifications,  field: "Qua
}

public Teacher(Teacher other) {
    this(other.getId(), other.getFirstName(), other.getLastName(), ot
    specializations.addAll(other.specializations);
    for (TimeSlot slot : other.availability) {
```

Field availability of org.app.model.Teacher — 5 usages

Project Files — Usages

Teacher.java 22 for (TimeSlot slot : other.**availability**) {
Teacher.java 23 **availability**.add(new TimeSlot(slot));
Teacher.java 34 **availability**.add(new TimeSlot(slot));
Teacher.java 45 for (TimeSlot slot : **availability**) {
Teacher.java 59 for (TimeSlot slot : **availability**) {

org/app/model/Teacher.java

```
public void addAvailability(TimeSlot slot) { availability.add(new TimeSlot(slot)); }
```

Polymorphism

collections: iterated through the parent type

```
82 @    public List<String> findConflicts(Exam exam) throws SQLException {  2 usages
83          List<String> conflicts = new ArrayList<>();
84          String myRoom = exam.getRoomId();
85          List<String> myInstruments = exam.getInstrumentIds();
86
87          for (Activity other : DbLoader.loadActivities()) {
88              if (other.getId().equals(exam.getId())) continue;
89              if (other.getStatus() == Activity.Status.CANCELLED) continue;
90              if (!other.overlaps(exam.getStart(), exam.getDurationMinutes())) continue;
91
```

method calls: each object runs its own behavior at runtime

```
87          for (Activity other : DbLoader.loadActivities()) {
88              if (other.getId().equals(exam.getId())) continue;
89              if (other.getStatus() == Activity.Status.CANCELLED) continue;
90              if (!other.overlaps(exam.getStart(), exam.getDurationMinutes())) continue;
```

Polymorphism

Overridden method

```
93  public abstract String getKind();  4 usages  3 implementations
94
95  public abstract Activity copy();  no usages  3 im
96
97  @Override
98  public String toString() {
```

Choose Implementation of Activity.getKind() (3 found)

Exam (org.app.model)	Backend
Lesson (org.app.model)	Backend
Rental (org.app.model)	Backend

```
113     @Override  4 usages
114     public String getKind() { return "EXAM"; }
117
```

```
78      @Override  4 usages
79      public String getKind() { return "LESSON"; }
82
```

dispatch

```
92              if (myRoom != null && myRoom.equals(roomOf(other))) {
93                  conflicts.add("Room " + myRoom + " busy: " + other);
```

```
193         for (Activity activity : activities) {
194             System.out.println("\n" + activity.getId() + " | " + activity.getKind() + " | " + activity.getStatus());
195             System.out.println("Time: " + activity.getStart().format(format) + " -> " + activity.getEnd().format(format));
196
```

Resolves to the right version at runtime (dynamic dispatch)

Copy constructor

```
19 @    public Teacher(Teacher other) {
20          this(other.getId(), other.getFirstName(), other.getLastName(), other.hourlyRateCents, other.qualifications);
21          specializations.addAll(other.specializations);
22          for (TimeSlot slot : other.availability) {
23              availability.add(new TimeSlot(slot));
24          }
25      }
```

```
57      public List<TimeSlot> getAvailability() {  5 usages
58          List<TimeSlot> result = new ArrayList<>();
59          for (TimeSlot slot : availability) {
60              result.add(new TimeSlot(slot));
61          }
62          return result;
```

Returns a copy, internal state cannot be modified from the outside

Encapsulation

```
3   public class Instrument {
4
5       private final String id;  4 usages
6       private final String name;  4 usages
7       private final String type;  4 usages
8       private final long hourlyPriceCents;  3 usages
9
10      public Instrument(String id, String name, String type, long hourlyPriceCents) {
11          this.id = ModelChecks.id(id);
12          this.name = ModelChecks.text(name,  field: "Instrument name");
13          this.type = ModelChecks.type(type);
14          this.hourlyPriceCents = ModelChecks.price(hourlyPriceCents);
15      }

21      public String getId() { return id; }
```

1) private field

2) constructor validate

3) immutable: using only get

Testing and observed behavior

31 Unit Test

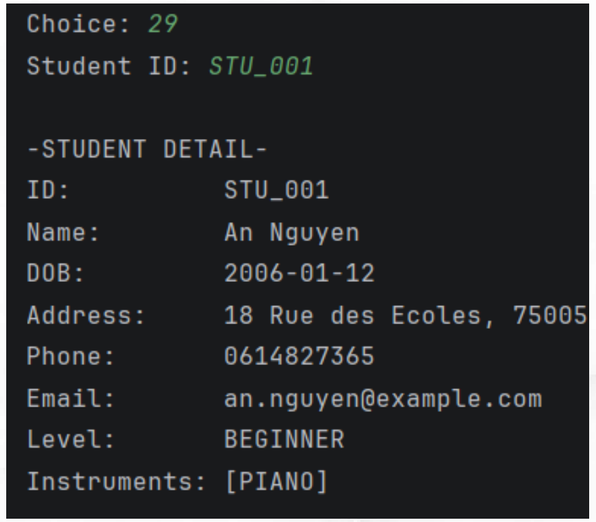
- Activity - Enrollment - Exam -Scheduling

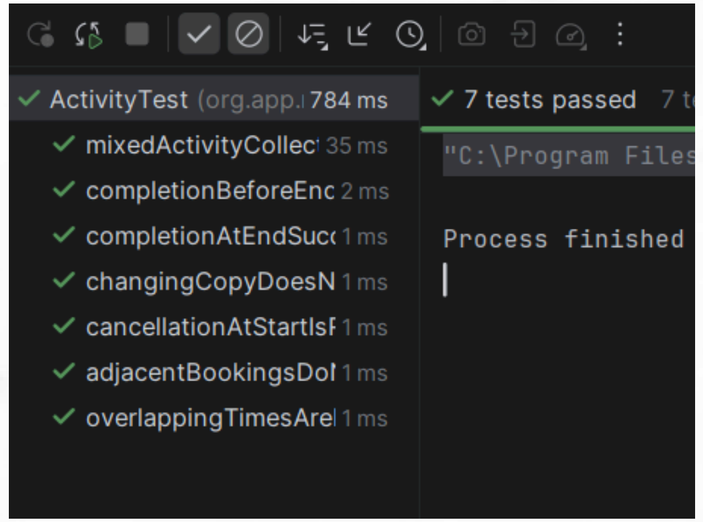
7 Demo Test Run

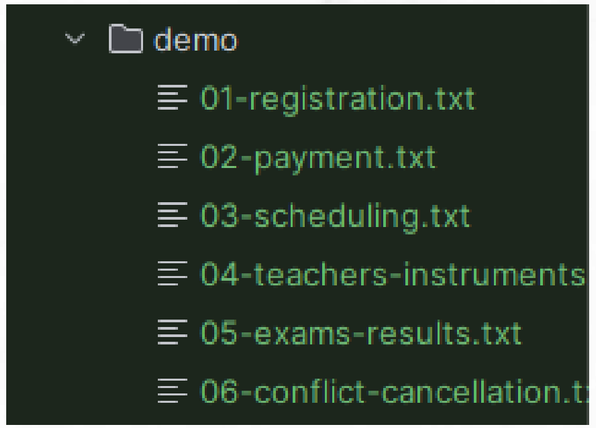
- Project demo requirement.

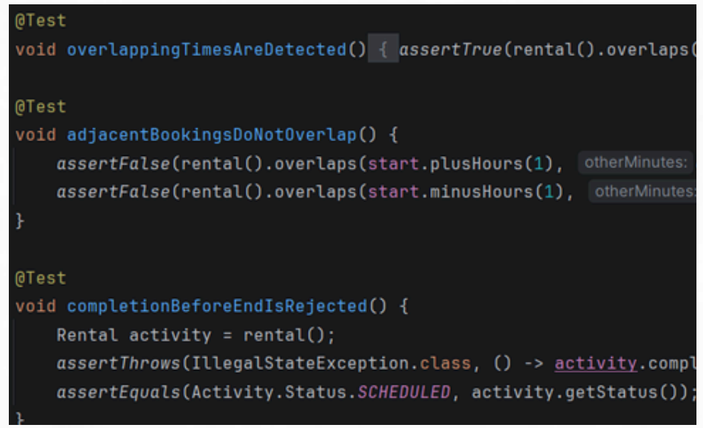
15 Manual Test Group

- Cancellation <24h consumes the lesson hour.
- handel conflict when scheduling, renting, booking
- charge - pay - balance calculation









Conclusion

1) Achievements

- Full console app + MySQL, 5 functional areas
- Bonus: Swing GUI & 31 unit tests
- OOP applied: abstraction, inheritance, polymorphism, copy constructors

2) Challenges & Solutions

Shared “activity” table for 2 peoples

➜ Made a single-table inheritance with **strict ownership** (WHERE kind='EXAM')

3) Future Improvements

- Web frontend
- User roles & access control